

SQL in Google Colab Using SQLite

Week 3

Mentor: Doris Mugah **Topic:** SQL Concepts & Relational Thinking

Environment: Google Colab

Database: SQLite

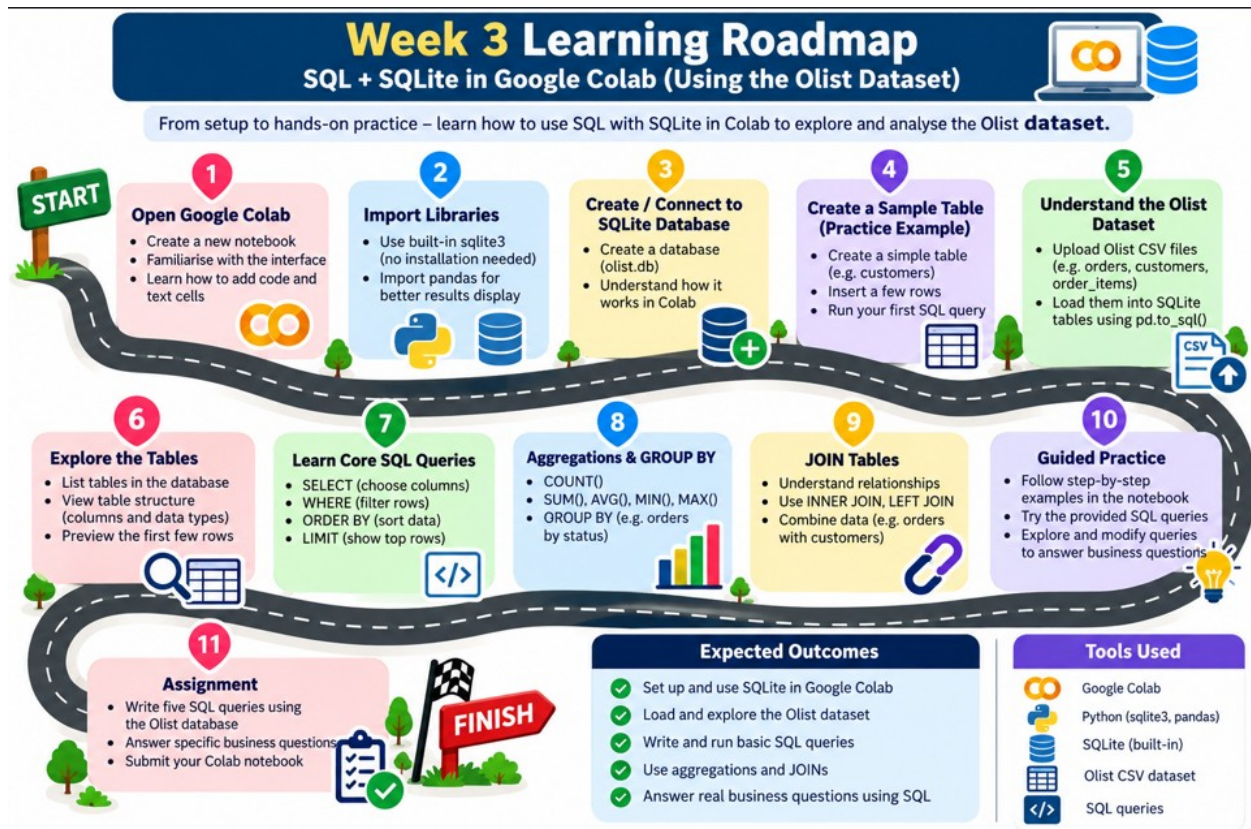
Why this approach?

You do **not** need to install MySQL, XAMPP, or DB Browser.

Python already includes support for SQLite through the `sqlite3` library.

In this notebook we will learn how to:

- Connect to SQLite
- Create a database
- Create a table
- Insert data
- Run `SELECT` queries
- Filter using `WHERE`
- Use Pandas to display SQL results
- Load an Olist CSV file
- Convert a Pandas DataFrame into an SQL table
- Practise `COUNT`, `GROUP BY`, and other SQL concepts



1. Import SQLite

SQLite support is already included with Python.

We only need to import the library.

```
import sqlite3
```

2. Create / Connect to a Database

The following line creates a database called `olist.db`.

If the database already exists, Python connects to it.

```
conn = sqlite3.connect("olist.db")  
print("Connected to SQLite successfully!")  
Connected to SQLite successfully!
```

What does `conn` mean?

`conn` means **connection**.

Think of it like this:

Google Colab to SQLite connection to olist.db

We use this connection whenever we want Python to communicate with our database.

3. Create a Cursor

A cursor allows us to send SQL commands to the database.

```
cursor = conn.cursor()
print('Cursor created successfully!')
Cursor created successfully!
```

4. Create Our First Table

Before using the full Olist dataset, we will create a very small `customers` table.

It will contain:

- `customer_id`
- `customer_name`
- `city`

Notice that `customer_id` is our **PRIMARY KEY** because it uniquely identifies each customer.

```
cursor.execute("DROP TABLE IF EXISTS customers")

cursor.execute("""
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT,
    city TEXT
)
""")

conn.commit()

print("Customers table created!")
Customers table created!
```

5. Insert Data

Now we will add three customers to the table.

```
cursor.execute("""
INSERT INTO customers
VALUES (1, 'Mary', 'Nairobi')
""")

cursor.execute("""
INSERT INTO customers
VALUES (2, 'John', 'Nakuru')
""")

cursor.execute("""
INSERT INTO customers
VALUES (3, 'Faith', 'Mombasa')
""")

conn.commit()

print("Data inserted successfully!")

Data inserted successfully!
```

6. Your First SELECT Query

`SELECT` is used to retrieve data from a table.

The `*` means:

Show me all columns.

```
cursor.execute("""
SELECT *
FROM customers
""")

results = cursor.fetchall()

results

[(1, 'Mary', 'Nairobi'), (2, 'John', 'Nakuru'), (3, 'Faith', 'Mombasa')]
```

7. Display SQL Results Using Pandas

The result above works, but it is not very easy to read.

Because we already know some Pandas, we can display our SQL results as a DataFrame.

```
import pandas as pd

query = """
SELECT *
FROM customers
"""

pd.read_sql_query(query, conn)

{"summary":{"name": "pd", "rows": 3, "fields": [{"column": "customer_id", "properties": {"dtype": "number", "std": 1, "min": 1, "max": 3, "num_unique_values": 3, "samples": [1, 2, 3]}, "semantic_type": "", "description": ""}, {"column": "customer_name", "properties": {"dtype": "string", "num_unique_values": 3, "samples": ["Mary", "John", "Faith"]}, "semantic_type": "", "description": ""}, {"column": "city", "properties": {"dtype": "string", "num_unique_values": 3, "samples": ["Nairobi", "Nakuru", "Mombasa"]}, "semantic_type": "", "description": ""}], "type": "dataframe"}
```

Much better!

`pd.read_sql_query()`:

1. Sends the SQL query to SQLite
2. Gets the result
3. Displays it as a Pandas DataFrame

8. SELECT Specific Columns

We do not always need every column.

Let's display only the customer's name and city.

```
query = """
SELECT customer_name, city
```

```

FROM customers
"""

pd.read_sql_query(query, conn)

{"summary":{"name": "pd", "rows": 3, "fields": [{"column": "customer_name", "properties": {"dtype": "string", "num_unique_values": 3, "samples": ["Mary", "John", "Faith"], "semantic_type": ""}, "description": ""}], "column": "city", "properties": {"dtype": "string", "num_unique_values": 3, "samples": ["Nairobi", "Nakuru", "Mombasa"], "semantic_type": ""}, "description": ""}], "type": "dataframe"}

```

9. Filtering with WHERE

Business Question

Which customers are from Nairobi?

We use **WHERE** when we want to filter records.

```

query = """
SELECT *
FROM customers
WHERE city = 'Nairobi'
"""

pd.read_sql_query(query, conn)

{"summary":{"name": "pd", "rows": 1, "fields": [{"column": "customer_id", "properties": {"dtype": "number", "std": null, "min": 1, "max": 1, "num_unique_values": 1, "samples": [1], "semantic_type": ""}, "description": ""}], "column": "customer_name", "properties": {"dtype": "string", "num_unique_values": 1, "samples": ["Mary"], "semantic_type": ""}, "column": "city", "properties": {"dtype": "string", "num_unique_values": 1, "samples": ["Nairobi"], "semantic_type": ""}, "description": ""}], "type": "dataframe"}

```

10. Practice

Try changing 'Nairobi' to 'Nakuru'.

What result do you get?

```
query = """
SELECT *
FROM customers
WHERE city = 'Nakuru'
"""

pd.read_sql_query(query, conn)

{"summary": "{\n  \"name\": \"pd\",\n  \"rows\": 1,\n  \"fields\": [\n    {\n      \"column\": \"customer_id\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": 2,\n        \"max\": 2,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          2\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"customer_name\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"John\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"city\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"Nakuru\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\"}
```

11. COUNT Records

Question

How many customers are in our table?

We use `COUNT()` to count records.

```
query = """
SELECT COUNT(*) AS total_customers
FROM customers
"""

pd.read_sql_query(query, conn)

{"summary": "{\n  \"name\": \"pd\",\n  \"rows\": 1,\n  \"fields\": [\n    {\n      \"column\": \"total_customers\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": 3,\n        \"max\": 3,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          3\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\"}
```

```
[\\n          3\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n          \\\"description\\\": \\\"\\\"\\n          }\\n          }\\n          ]\\n}\\\", \"type\": \"dataframe\"}
```

12. Moving to the Olist Dataset

Now that we understand the basic process, we can use the real Olist data.

The workflow will be:

CSV file → Pandas → SQLite → SQL Query → Results

For example, upload:

`olist_orders_dataset.csv`

to your Google Colab session.

13. Upload the Olist Orders CSV

Run the cell below.

Click **Choose Files** and select:

`olist_orders_dataset.csv`

```
from google.colab import files
uploaded = files.upload()
<IPython.core.display.HTML object>
Saving olist_orders_dataset.csv to olist_orders_dataset.csv
```

14. Load the CSV with Pandas

After uploading the file, read it using Pandas.

```
orders = pd.read_csv("olist_orders_dataset.csv")
orders.head()

{"summary":{"\\n  \\\"name\\\": \\\"orders\\\",\\n  \\\"rows\\\": 99441,\\n
\\\"fields\\\": [\\n    {\\n      \\\"column\\\": \\\"order_id\\\",\\n
\\\"properties\\\": {\\n        \\\"dtype\\\": \\\"string\\\",\\n
\\\"num_unique_values\\\": 99441,\\n        \\\"samples\\\": [\\n
\\\"b9a6c5f5df52c7226ac85aee7524c27f\\\",\\n
```



```
orders.shape
(99441, 8)
orders.columns
Index(['order_id', 'customer_id', 'order_status',
      'order_purchase_timestamp',
      'order_approved_at', 'order_delivered_carrier_date',
      'order_delivered_customer_date',
      'order_estimated_delivery_date'],
      dtype='object')
```

16. Convert the Pandas DataFrame to an SQL Table

This is the important bridge between Pandas and SQL.

We will take our `orders` DataFrame and save it inside SQLite as a table called `orders`.

```
orders.to_sql(
    "orders",
    conn,
    if_exists="replace",
    index=False
)

print("Orders table created in SQLite!")
Orders table created in SQLite!
```

17. Query the Olist Orders Table

Question

Show the first 10 orders.

```
query = """
SELECT *
FROM orders
LIMIT 10
"""

pd.read_sql_query(query, conn)
```

```

{"summary":{"\n  \"name\": \"pd\", \n  \"rows\": 10, \n  \"fields\": [\n    {\n      \"column\": \"order_id\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 10, \n        \"samples\": [\n          \"76c6e866289321a7c93b82b54852dc33\", \n          \"53cdb2fc8bc7dce0b6741e2150273451\", \n          \"a4591c265e18cb1dcee52889e2d8acc3\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      {\n        \"column\": \"customer_id\", \n        \"properties\": {\n          \"dtype\": \"string\", \n          \"num_unique_values\": 10, \n          \"samples\": [\n            \"f54a9f0e6b351c431402b8461ea51999\", \n            \"b0830fb4747a6c6d20dea0b8c802d7ef\", \n            \"503740e9ca751ccdda7ba28e9ab8f608\" \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        }, \n        {\n          \"column\": \"order_status\", \n          \"properties\": {\n            \"dtype\": \"category\", \n            \"num_unique_values\": 2, \n            \"samples\": [\n              \"invoiced\", \n              \"delivered\" \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\" \n          }, \n          {\n            \"column\": \"order_purchase_timestamp\", \n            \"properties\": {\n              \"dtype\": \"object\", \n              \"num_unique_values\": 10, \n              \"samples\": [\n                \"2017-01-23 18:29:09\", \n                \"2018-07-24 20:41:37\" \n              ], \n              \"semantic_type\": \"\", \n              \"description\": \"\" \n            }, \n            {\n              \"column\": \"order_approved_at\", \n              \"properties\": {\n                \"dtype\": \"object\", \n                \"num_unique_values\": 10, \n                \"samples\": [\n                  \"2017-01-25 02:50:47\", \n                  \"2018-07-26 03:24:27\" \n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\" \n              }, \n              {\n                \"column\": \"order_delivered_carrier_date\", \n                \"properties\": {\n                  \"dtype\": \"object\", \n                  \"num_unique_values\": 9, \n                  \"samples\": [\n                    \"2017-01-26 14:16:31\", \n                    \"2018-07-26 14:31:00\" \n                  ], \n                  \"semantic_type\": \"\", \n                  \"description\": \"\" \n                }, \n                {\n                  \"column\": \"order_delivered_customer_date\", \n                  \"properties\": {\n                    \"dtype\": \"object\", \n                    \"num_unique_values\": 9, \n                    \"samples\": [\n                      \"2017-02-02 14:08:10\", \n                      \"2018-08-07 15:27:45\" \n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\" \n                  }, \n                  {\n                    \"column\": \"order_estimated_delivery_date\", \n                    \"properties\": {\n                      \"dtype\": \"object\", \n                      \"num_unique_values\": 10, \n                      \"samples\": [\n                        \"2017-03-06 00:00:00\", \n                        \"2018-08-13 00:00:00\" \n                      ], \n                      \"semantic_type\": \"\", \n                      \"description\": \"\" \n                    } \n                  } \n                ] \n              }, \n              \"type\": \"dataframe\" \n            } \n          ] \n        } \n      } \n    } \n  ] \n}

```

18. Select Specific Columns

Let's display only:

- order ID
- customer ID
- order status

```
query = """
SELECT order_id, customer_id, order_status
FROM orders
LIMIT 10
"""

pd.read_sql_query(query, conn)

{"summary":{"\n  \"name\": \"pd\",\n  \"rows\": 10,\n  \"fields\": [\n    {\n      \"column\": \"order_id\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 10,\n        \"samples\": [\n          \"76c6e866289321a7c93b82b54852dc33\",\n          \"53cdb2fc8bc7dce0b6741e2150273451\",\n          \"a4591c265e18cb1dcee52889e2d8acc3\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"customer_id\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 10,\n        \"samples\": [\n          \"f54a9f0e6b351c431402b8461ea51999\",\n          \"b0830fb4747a6c6d20dea0b8c802d7ef\",\n          \"503740e9ca751ccdda7ba28e9ab8f608\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"order_status\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"invoiced\",\n          \"delivered\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}, \"type\": \"dataframe\"}
```

19. WHERE with Olist

Business Question

Which orders were delivered?

```
query = """
SELECT *
FROM orders
WHERE order_status = 'delivered'
"""
```

```
LIMIT 10
"""
```

```
pd.read_sql_query(query, conn)
```

```
{"summary":{"\n  \"name\": \"pd\",\n  \"rows\": 10,\n  \"fields\": [\n    {\n      \"column\": \"order_id\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 10,\n        \"samples\": [\n          \"e69bfb5eb88e0ed6a785585b27e16dbf\",\n          \"53cdb2fc8bc7dce0b6741e2150273451\",\n          \"a4591c265e18cb1dcee52889e2d8acc3\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      {\n        \"column\": \"customer_id\",\n        \"properties\": {\n          \"dtype\": \"string\",\n          \"num_unique_values\": 10,\n          \"samples\": [\n            \"31ad1d1b63eb9962463f764d4e6e0c9d\",\n            \"b0830fb4747a6c6d20dea0b8c802d7ef\",\n            \"503740e9ca751ccdda7ba28e9ab8f608\"\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        {\n          \"column\": \"order_status\",\n          \"properties\": {\n            \"dtype\": \"category\",\n            \"num_unique_values\": 1,\n            \"samples\": [\n              \"delivered\"\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          },\n          {\n            \"column\": \"order_purchase_timestamp\",\n            \"properties\": {\n              \"dtype\": \"object\",\n              \"num_unique_values\": 10,\n              \"samples\": [\n                \"2017-07-29 11:55:02\"\n              ],\n              \"semantic_type\": \"\",\n              \"description\": \"\"\n            },\n            {\n              \"column\": \"order_approved_at\",\n              \"properties\": {\n                \"dtype\": \"object\",\n                \"num_unique_values\": 10,\n                \"samples\": [\n                  \"2017-07-29 12:05:32\"\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n              },\n              {\n                \"column\": \"order_delivered_carrier_date\",\n                \"properties\": {\n                  \"dtype\": \"object\",\n                  \"num_unique_values\": 10,\n                  \"samples\": [\n                    \"2017-08-10 19:45:24\"\n                  ],\n                  \"semantic_type\": \"\",\n                  \"description\": \"\"\n                },\n                {\n                  \"column\": \"order_delivered_customer_date\",\n                  \"properties\": {\n                    \"dtype\": \"object\",\n                    \"num_unique_values\": 10,\n                    \"samples\": [\n                      \"2017-08-16 17:14:30\"\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\"\n                  },\n                  {\n                    \"column\": \"order_estimated_delivery_date\",\n                    \"properties\": {\n                      \"dtype\": \"object\",\n                      \"num_unique_values\": 9,\n                      \"samples\": [\n                        \"2017-03-06 00:00:00\"\n                      ],\n                      \"semantic_type\": \"\",\n                      \"description\": \"\"\n                    }\n                  }\n                }\n              }\n            ]\n          }\n        }\n      },\n      \"type\": \"dataframe\"}
```

20. Count All Orders

Business Question

How many orders are in the dataset?

```
query = """
SELECT COUNT(*) AS total_orders
FROM orders
"""

pd.read_sql_query(query, conn)

{"summary": "{\n  \"name\": \"pd\",\n  \"rows\": 1,\n  \"fields\": [\n    {\n      \"column\": \"total_orders\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": null,\n        \"min\": 99441,\n        \"max\": 99441,\n        \"num_unique_values\": 1,\n        \"samples\": [\n          99441\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}", "type": "dataframe"}
```

21. GROUP BY

Business Question

How many orders are there for each order status?

GROUP BY places records with the same value into groups.

```
query = """
SELECT
    order_status,
    COUNT(*) AS number_of_orders
FROM orders
GROUP BY order_status
"""

pd.read_sql_query(query, conn)

{"summary": "{\n  \"name\": \"pd\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"order_status\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 8,\n        \"samples\": [\n          \"canceled\",\n          \"processing\",\n          \"approved\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"number_of_orders\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 33962,\n        \"min\": 2,\n        \"max\": 33962,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          2,\n          3,\n          4,\n          5,\n          6,\n          7,\n          8,\n          9\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}", "type": "dataframe"}
```

```

{"max\\": 96478,\\n          \\\"num_unique_values\\\": 8,\\n
\\\"samples\\\": [\\n          625,\\n          301,\\n          2\\n
\\n          ],\\n          \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n          }\\n          }\\n          ]\\n}\\", \"type\": \"dataframe\"}

```

22. ORDER BY

We can sort our grouped results.

Let's show the order statuses from the largest number of orders to the smallest.

```

query = """
SELECT
    order_status,
    COUNT(*) AS number_of_orders
FROM orders
GROUP BY order_status
ORDER BY number_of_orders DESC
"""

pd.read_sql_query(query, conn)

{"summary": "{\\n  \\\"name\\\": \\\"pd\\\",\\n  \\\"rows\\\": 8,\\n  \\\"fields\\\": [\\n
\\n    \\\"column\\\": \\\"order_status\\\",\\n    \\\"properties\\\": {\\n
\\\"dtype\\\": \\\"string\\\",\\n    \\\"num_unique_values\\\": 8,\\n
\\\"samples\\\": [\\n    \\\"shipped\\\",\\n    \\\"processing\\\",\\n
\\\"delivered\\\"\\n    ],\\n    \\\"semantic_type\\\": \\\"\\\",\\n
\\\"description\\\": \\\"\\\"\\n    }\\n    },\\n    {\\n    \\\"column\\\":
\\\"number_of_orders\\\",\\n    \\\"properties\\\": {\\n    \\\"dtype\\\":
\\\"number\\\",\\n    \\\"std\\\": 33962,\\n    \\\"min\\\": 2,\\n
\\\"max\\\": 96478,\\n    \\\"num_unique_values\\\": 8,\\n
\\\"samples\\\": [\\n    1107,\\n    301,\\n    96478\\n
],\\n    \\\"semantic_type\\\": \\\"\\\",\\n    \\\"description\\\": \\\"\\\"\\n
}\\n    }\\n    ]\\n}\\", \"type\": \"dataframe\"}

```

23. SQL vs Pandas

Notice that you already know many of these ideas from Pandas.

Task	SQL	Pandas
Select columns	SELECT	df[['column']]
Filter	WHERE	df[df['column'] == value]
Group	GROUP BY	.groupby()
Count	COUNT()	.count() / .size()

Task	SQL	Pandas
Average	<code>AVG()</code>	<code>.mean()</code>
Sort	<code>ORDER BY</code>	<code>.sort_values()</code>
Join tables	<code>JOIN</code>	<code>.merge()</code>

The syntax is different, but the **data thinking is similar**.

Key Takeaways

Today we learned that:

- We can practise SQL in Google Colab without installing database software.
- Python includes SQLite support through `sqlite3`.
- `sqlite3.connect()` connects us to a database.
- `SELECT` retrieves data.
- `WHERE` filters data.
- `COUNT()` counts records.
- `GROUP BY` summarizes categories.
- `ORDER BY` sorts results.
- Pandas DataFrames can be converted into SQLite tables using `.to_sql()`.
- SQL results can be displayed using `pd.read_sql_query()`.

Remember

Start with the business question, identify the table and columns you need, and then write the SQL.